

H.W
19th June, 2026

Tutorial 3

Date:

Page:

- 1) A university wants a system to store student information and display admission details.

Class student {

private string name;

private int rollNo;

private string faculty;

// Constructor

public student (string name, int rollNo, string faculty) {

this.name = name;

this.rollNo = rollNo;

this.faculty = faculty;

}

// getters

public string getName () {

return name;

}

public int getRollNo () {

return rollNo;

}

public string getFaculty () {

return faculty;

}

// setters

Public void setName (String name) {

2. this.name = name;

Public void setRollNo (int rollNo) {

2. this.rollNo = rollNo;

Public void setFaculty (String faculty) {

2. this.faculty = faculty;

Public void displayPtoFile () {

System.out.println ("Name: " + name);

System.out.println ("RollNo: " + rollNo);

2. System.out.println ("Faculty: " + faculty);

Public static void main (String [] args) {

Student s = new Student ("Admin", 101, "BCA");

2. s.displayPtoFile ();

2.

2) Employee, Teacher and AdminStaff (Inheritance & Method overriding)

Class Employee {

String name;

Employee (String name) {

2. this.name = name;

2. public void displayDetails() {
system.out.println("Employee Name: " + name);
2

Class Teacher extends Employee {

Teacher (setting name) {
super(name);
2

@ override

2. public void displayDetails() {
system.out.println("Teaching staff: " + name);
2

Class AdminStaff extends Employee {

AdminStaff (setting name) {
super(name);
2

@ override

2. public void displayDetails() {
system.out.println("Administrative staff: " + name);
2

public class main {

public static void main (String args) {

Employee t = new Teacher ("Ravi");

Employee a = new AdminStaff ("Shyam");

t.displayDetails();

a.displayDetails();

}

3) Student can register for multiple courses.

import java.util.ArrayList;

class course {

String CourseName;

course (String CourseName) {

this.CourseName = CourseName;

}

class student {

String name;

ArrayList <course> courses = new ArrayList
();

student (String name) {

this.name = name;

}

Public void addCourse (course c) {
 courses.add(c);
}

public void displayCourses () {
 System.out.println ("Registered Courses:");
 for (Course c : courses) {
 System.out.println (c.getCourseName());
 }
}

public static void main (String args[]) {
 Student s = new Student ("Adnan");
}

s.addCourse (new Course ("Java"));
s.addCourse (new Course ("Database"));
s.addCourse (new Course ("Networking"));

s.displayCourses();
}

4. Different students have different fee structures.

abstract class student {
 String name;

 Student (String name) {
 this.name = name;

 }

2. abstract void calculateFee();

class undergraduate extends student {

undergraduate (String name) {

super (name);

2. @Override

void calculateFee () {

system.out.println ("undergraduate Fee
= Rs 50,000");

2.

class Graduate extends student {

Graduate (String name) {

super (name);

2.

@Override

void calculateFee () {

system.out.println ("Graduate Fee = Rs.
80,000");

2.

2.

public class Main {

public static void main (String[] args) {

Student s1 = new Undergraduate("Ram");
Student s2 = new Graduate("Hat");

s1.calculateFee();
s2.calculateFee();

5. Different departments calculate grades differently.

interface ResultProcessor {

void calculateGrade(int marks);

class EngineeringDepartment implements
ResultProcessor {

@Override

public void calculateGrade(int marks) {

if (marks >= 80)

system.out.println("Engineering Grade: A");

else if (marks >= 60)

system.out.println("Engineering Grade: B");

else

system.out.println("Engineering Grade: C");

class ManagementDepartment implements

ResultProcessor &

@Override

public void CalculateGrade (int marks) &

if (marks >= 75)

System.out.println ("Management Grade
is A");

else if (marks >= 55)

System.out.println ("Management
Grade is B");

else

System.out.println ("Management Grade
is C");

}

public class Main &

public static void main (String[] args) &

ResultProcessor obj = new Engineering
Department();

ResultProcessor obj2 = new Management
Department();

obj.CalculateGrade (85);

obj2.CalculateGrade (70);

}

}

6. Library members can be students or

Teachers:

class Member {

String name;

Member (String name) {

this.name = name;

}

public void borrowingLimit () {

System.out.println ("General borrowing
limit ' '");

}

}

class StudentMember extends Member {

StudentMember (String name) {

super (name);

}

@Override

public void borrowingLimit () {

System.out.println ("Student can borrow
3 books.");

}

}

class TeacherMember extends Member {

TeacherMember (String name) {

super (name);

@ override

public void borrowingLimit() {

System.out.println("Teacher can borrow
10 books.");

}

public class Main {

public static void main (String [] args) {

Member s = new StudentMember ("Ram");
Member t = new TeacherMember ("Hadi");

s.borrowingLimit();

t.borrowingLimit();

}

7. Attendance percentage calculation differs
among faculties.

class Attendance {

public void calculateAttendance (int attended,
int total) {

double percentage = (attended * 100.0) / total;
System.out.println("Attendance: " + percentage
+ "%");

}

}

class EngineeringAttendance extends Attendance {

@Override

public void calculateAttendance(int attendance,
int total) {

double percentage = (attendance * 100.0) / total; {

System.out.println("Engineering Attendance: " +
percentage + "%");

}

}

class MedicalAttendance extends Attendance {

@Override

public void calculateAttendance(int attended,
int total) {

double percentage = (attended * 100.0) / total;

System.out.println("Medical Attendance: " +
percentage + "%");

}

}

public class Main {

public static void main (String[] args) {

Attendance eng = new

EngineeringAttendance();

Attendance med = new MedicalAttendance
();

eng.calculateAttendance (42, 50);

med.calculateAttendance (45, 50);

2.
2. Notifications can be sent through Email
or SMS.

2.
2. interface Notification {
void sendMessage (String message);
}

2.
2. class EmailNotification implements Notification {

2.
2. @Override
public void sendMessage (String message) {
System.out.println ("Email sent: " +
message);
}

2.
2. class SMSNotification implements Notification {

2.
2. @Override
public void sendMessage (String message) {
System.out.println ("SMS sent: " +
message);
}

2.
2. public class Main {


```

public static void main (String [] args) {
    Notification email = new
    EmailNotification ();
    Notification sms = new SMSNotification
    ();

    email.sendMessage ("Exam starts tomorrow.
    ow.");
    sms.sendMessage ("fee deadline is next.");
}
}

```

9. Every student gets a unique ID.

```

class Student {
    private static int counter = 1000;
    private int studentId;
    private String name;

    Student (String name) {
        this.name = name;
        studentId = ++counter;
    }

    public void display () {
        System.out.println ("Student ID: " +
        studentId);
        System.out.println ("Name: " + name);
    }
}

```


public static void main (String [] args) {
 Student s1 = new student ("Ram");
 Student s2 = new student ("Hadi");

s1.display();

s2.display();

}

}

20. Scholarship eligibility differs among pds -
gams.

abstract class Scholarship {

abstract void checkEligibility();

}

class MeritScholarship extends Scholarship {

@Override

void checkEligibility() {

System.out.println ("Eligible if GPA
 >= 7.8");

}

}

class NeedBasedScholarship extends Scholarship {

@Override

Date: _____

Page: _____

```
void checkEligibility() {  
    System.out.println("Eligible based on  
    financial need.");  
}
```

```
public class Main {  
    public static void main (String[] args) {  
        Scholarship sa = new MedScholarship();  
        Scholarship sb = new needBasedScholarship  
        ();  
    }  
}
```

```
sa.checkEligibility();  
sb.checkEligibility();
```

